Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Sistemas de Bases de Datos 2

MANUAL DE USUARIO

Sistema de Gestión de Reservas y Boletos Aéreos con Apache Cassandra

Proyecto 1 — Segundo semestre 2026

Victor Ronaldo Gómez Lara
Carné: 201114493

Guatemala, septiembre de 2026

Índice

1. Propósito del manual

Este manual explica cómo instalar el clúster de Apache Cassandra del sistema de reservas aéreas, cargar los datos y ejecutar las consultas del negocio. Está dirigido a una persona con conocimientos básicos de línea de comandos que necesite reproducir el entorno desde cero o consultar la base de datos ya desplegada.

Las instrucciones se redactaron para Ubuntu. En otras distribuciones de Linux o en macOS los comandos son equivalentes; en Windows se recomienda ejecutarlos dentro de WSL, ya que varios de ellos emplean sintaxis de shell no disponible en PowerShell.

2. Requisitos previos

Componente	Versión	Verificación
Docker Engine	20.10 o superior	`docker --version`
Docker Compose	v2 (plugin)	`docker compose version`
Python	3.10 o superior	`python3 --version`
Memoria disponible	4 GB libres como mínimo	`free -h`
Espacio en disco	3 GB libres	`df -h`

El usuario debe pertenecer al grupo docker para ejecutar los comandos sin privilegios de administrador. Si no es el caso:

```
sudo usermod -aG docker $USER
newgrp docker
docker ps
```

Si el último comando responde sin error de permisos, la configuración es correcta. No se recomienda ejecutar el proyecto como usuario root: los archivos generados quedarían con propietario root y tanto el entorno virtual de Python como las operaciones de Git presentarían problemas de permisos.

3. Estructura del proyecto

El repositorio se organiza de la siguiente forma:

```
BD2_2S26_Proyectos_201114493/
└── Primer Proyecto/
    ├── docker-compose.yml      Definición del clúster de 3 nodos
    ├── cql/
    │   ├── 01_keyspace.cql     Creación del keyspace
    │   ├── 02_tablas.cql       Tablas de entidad
    │   ├── 03_tablas_queries.cql  Tablas especializadas por consulta
    │   └── 04_consultas.cql    Las 5 consultas del negocio
    ├── scripts/
    │   ├── requirements.txt    Dependencias de Python
```

```
    │   ├── carga_datos.py      Carga masiva con Batch Writes
    │   ├── consultas.py        Ejecución de las 5 consultas
    │   ├── consultas_medicion.py  Medición de latencias
    │   └── pruebas_fallos.py   Comparación de Consistency Levels
    └── docs/                   Evidencias y documentación
```

4. Instalación del clúster

4.1 Levantar los nodos

Desde la carpeta Primer Proyecto:

```
docker compose up -d
```

El arranque es secuencial: cass2 espera a que cass1 esté operativo y cass3 espera a cass2. El proceso completo tarda entre cinco y ocho minutos. Para seguir el avance:

```
watch -n 15 'docker exec cass1 nodetool status 2>/dev/null'
```

No debe continuarse hasta que aparezcan tres líneas que comiencen con UN (Up/Normal). Se sale del monitoreo con Ctrl+C.

```
Datacenter: dc1
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
--  Address     Load       Tokens  Owns (effective)  Rack
UN  172.19.0.2  32.81 MiB  16      100.0%            rack1
UN  172.19.0.4  32.74 MiB  16      100.0%            rack1
UN  172.19.0.3  32.81 MiB  16      100.0%            rack1
```

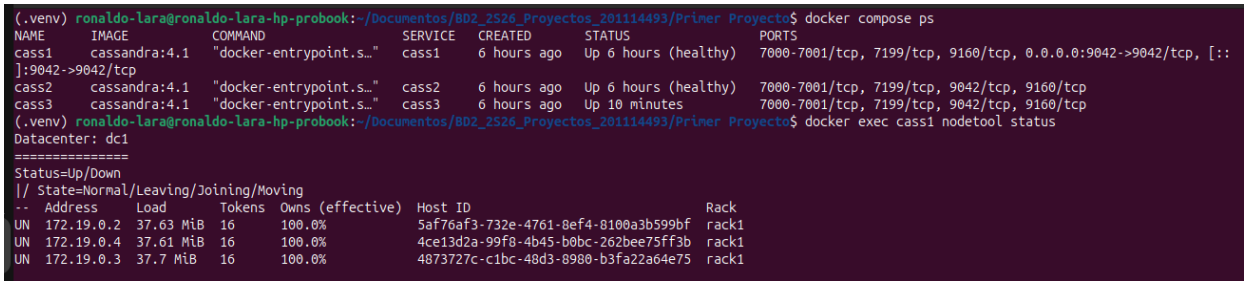


Figura 1. Contenedores activos y anillo formado con los tres nodos en estado UN.

4.2 Crear el keyspace y las tablas

Los archivos CQL se montan dentro del contenedor cass1 en la ruta /cql. Se ejecutan en orden, con un tiempo de espera ampliado porque las operaciones de definición de esquema requieren acuerdo entre los tres nodos:

```
docker exec -i cass1 cqlsh --request-timeout=120 -f /cql/01_keyspace.cql
docker exec -i cass1 cqlsh --request-timeout=120 -f /cql/02_tablas.cql
docker exec -i cass1 cqlsh --request-timeout=120 -f /cql/03_tablas_queries.cql
```

Entre la ejecución de un archivo y el siguiente conviene verificar que el esquema se haya propagado. El siguiente comando debe mostrar un único identificador asociado a las tres direcciones IP:

```
docker exec cass1 nodetool describecluster | grep -A3 'Schema versions'
```

Si aparecen dos o más identificadores distintos, debe esperarse treinta segundos y repetir la comprobación antes de continuar. Finalmente se verifica la creación de las quince tablas:

```
docker exec cass1 cqlsh -e "USE aerolinea; DESCRIBE TABLES;"
```

Advertencia: el archivo 01_keyspace.cql contiene la instrucción DROP KEYSPACE IF EXISTS al inicio. Ejecutarlo nuevamente elimina la totalidad de los datos cargados.

4.3 Preparar el entorno de Python

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r scripts/requirements.txt
```

Para comprobar que el controlador se comunica con el clúster:

```
python3 -c "from cassandra.cluster import Cluster;
c=Cluster(['127.0.0.1']); s=c.connect('aerolinea');
print(s.execute('SELECT release_version FROM system.local').one())"
```

5. Carga de datos

Con el entorno virtual activado:

```
python3 scripts/carga_datos.py
```

El script genera 25,000 pasajeros, 40 aeronaves y 520 vuelos con sus asientos, reservas y pagos, alcanzando 100,080 reservas. La ejecución tarda aproximadamente dos minutos e informa del avance cada veinticinco vuelos.

```
Insertando 25000 pasajeros...
  OK (3.5s)
Generando 520 vuelos con sus reservas...
  vuelos 25/520 - reservas: 4789 (8s)
  ...
Reservas insertadas: 100080
Carga finalizada en 105.1s
```

Los parámetros del volumen de datos se encuentran al inicio del archivo y pueden modificarse:

Parámetro	Valor	Efecto
`CANT_VUELOS`	520	Determina el total de reservas (unas 192 por vuelo)
`CANT_PASAJEROS`	25000	Universo de pasajeros disponibles
`TAM_BATCH`	25	Sentencias por lote; valores mayores pueden causar WriteTimeout
`MAX_PENDIENTES`	8	Lotes asíncronos simultáneos

Verificación de la carga y de su distribución entre nodos:

```
docker exec cass1 nodetool status aerolinea
docker exec cass1 cqlsh --request-timeout=180 \
  -e "SELECT COUNT(*) FROM aerolinea.reservas_por_id;"
```

La columna Load debe mostrar valores similares en los tres nodos. El conteo emite la advertencia «Aggregation query used without partition key», que es esperada: se trata de una verificación puntual, no de una consulta del sistema.

Para repetir la carga es indispensable vaciar antes todas las tablas. De lo contrario, las columnas de tipo counter acumulan los valores de ambas ejecuciones y las cifras de disponibilidad e ingresos quedan duplicadas.

```
for t in pasajeros_por_id aeronaves_por_id vuelos_por_id \
  asientos_por_vuelo reservas_por_id pagos_por_reserva \
  disponibilidad_por_clase capacidad_por_clase \
  reservas_por_pasajero manifiesto_por_vuelo \
  reservas_por_ruta_mes vuelos_por_ruta_mes \
  ingresos_por_vuelo top_ingresos_por_mes \
  reservas_pendientes_por_vuelo; do
```

```
  docker exec cass1 cqlsh --request-timeout=180 \
    -e "TRUNCATE aerolinea.$t;"
done
```

6. Ejecución de las consultas

Las cinco consultas pueden ejecutarse de dos formas. La primera, mediante el script que las corre todas con parámetros tomados automáticamente del conjunto de datos:

```
python3 scripts/consultas.py
```

La segunda, de forma individual desde cqlsh. Para ello se requiere un identificador válido, que puede obtenerse así:

```
docker exec cass1 cqlsh -e \
  "SELECT vuelo_id, codigo_vuelo, origen, destino
   FROM aerolinea.vuelos_por_id LIMIT 5;"
```

Las secciones siguientes documentan cada consulta con sus parámetros y el resultado esperado. Los identificadores mostrados corresponden al conjunto de datos de referencia; al recargar los datos se generan identificadores distintos.

6.1 Q1 — Disponibilidad de asientos por clase

Devuelve el número de asientos ocupados y disponibles de un vuelo, desglosado por clase. Parámetro: el identificador del vuelo.

```
SELECT clase, asientos_ocupados, asientos_disponibles
FROM aerolinea.disponibilidad_por_clase
WHERE vuelo_id = 5e8120ae-9633-4f76-890a-fd3502e41bd9;
```

Resultado esperado — tres filas, una por clase:

```
 clase     | asientos_ocupados | asientos_disponibles
 economica |               131 |                   67
 ejecutiva |                27 |                    3
 primera   |                 6 |                    6
```

La capacidad nominal de cada clase se consulta en la tabla complementaria, ya que una tabla con columnas de tipo counter no admite columnas de otro tipo:

```
SELECT clase, capacidad_clase FROM aerolinea.capacidad_por_clase
WHERE vuelo_id = 5e8120ae-9633-4f76-890a-fd3502e41bd9;
```

6.2 Q2 — Historial cronológico de un pasajero

Devuelve las reservas de un pasajero dentro de un rango de fechas, ordenadas de la más reciente a la más antigua, con el estado del pago incluido. Parámetros: identificador del pasajero, fecha inicial y fecha final.

```
SELECT fecha_reserva, codigo_vuelo, origen, destino,
       numero_asiento, clase, estado_reserva, estado_pago, monto
FROM aerolinea.reservas_por_pasajero
WHERE pasajero_id = 8d92b228-e768-4fa8-8090-39af7b49df67
  AND fecha_reserva >= '2026-01-01'
```

```
  AND fecha_reserva <= '2026-12-31';
```

Resultado esperado — las reservas del pasajero dentro del rango:

```
2026-09-20 | AV238 | BOG->GUA | 036B | confirmada | pagado | 394.33
2026-07-23 | AV162 | GUA->SJO | 042D | confirmada | pagado | 137.59
2026-06-15 | AV950 | YYZ->BOG | 001A | confirmada | pagado | 1225.31
```

Para obtener un identificador de pasajero con historial:

```
docker exec cass1 cqlsh -e \
  "SELECT pasajero_id FROM aerolinea.reservas_por_pasajero LIMIT 5;"
```

6.3 Q3 — Manifiesto de vuelo

Devuelve el listado de pasajeros de un vuelo ordenado por número de asiento, con clase, estado de la reserva y estado del pago. Parámetro: el identificador del vuelo.

```
SELECT numero_asiento, nombre_pasajero, documento, clase,
       estado_reserva, estado_pago
FROM aerolinea.manifiesto_por_vuelo
WHERE vuelo_id = 5e8120ae-9633-4f76-890a-fd3502e41bd9;
```

Resultado esperado — una fila por asiento ocupado, en orden ascendente:

```
001A | Jorge Aguilar Ramirez | primera | confirmada | pagado
001B | Sofia Mendoza Gomez   | primera | confirmada | pagado
001D | Camila Vasquez Lopez  | primera | confirmada | pagado
```

El vuelo de referencia devuelve 164 filas. Las filas se entregan ya ordenadas por el motor; no se requiere cláusula ORDER BY ni ordenamiento en el cliente.

6.4 Q4 — Porcentaje de ocupación por ruta y rango de fechas

Esta consulta se resuelve con dos lecturas sobre tablas que comparten la misma clave de partición. Parámetros: aeropuerto de origen, aeropuerto de destino y periodo en formato AAAA-MM.

```
SELECT vuelo_id, fecha_salida, reservas_confirmadas
FROM aerolinea.reservas_por_ruta_mes
WHERE origen = 'PTY' AND destino = 'LIM' AND anio_mes = '2026-07';

SELECT vuelo_id, codigo_vuelo, fecha_salida, capacidad
FROM aerolinea.vuelos_por_ruta_mes
WHERE origen = 'PTY' AND destino = 'LIM' AND anio_mes = '2026-07';
```

El porcentaje se obtiene dividiendo las reservas confirmadas entre la capacidad de cada vuelo:

```
AV663  2026-07-23  144/240 =  60.0% de ocupacion
AV313  2026-07-27  131/240 =  54.6% de ocupacion
```

El periodo forma parte de la clave de partición para evitar particiones de gran tamaño. Una consulta que abarque varios meses requiere una ejecución por cada mes del rango.

Los códigos de aeropuerto disponibles en el conjunto de datos son GUA, SAL, SJO, PTY, MEX, MIA, LAX, JFK, BOG, LIM, MAD, IAH, YYZ y SCL.

6.5 Q5 — Top N de vuelos por ingresos generados

Devuelve los vuelos con mayor ingreso de un periodo, ordenados de mayor a menor. Parámetros: periodo en formato AAAA-MM y número de resultados.

```
SELECT codigo_vuelo, origen, destino, monto_total, vuelo_id
FROM aerolinea.top_ingresos_por_mes
WHERE anio_mes = '2026-09'
LIMIT 10;
```

Resultado esperado — los diez vuelos de mayor ingreso del mes:

```
AV945 | SAL->LIM | 8988938
AV701 | IAH->LAX | 8324667
AV286 | PTY->MEX | 8243959
```

Los montos se expresan en centavos: el valor 8988938 corresponde a 89,889.38 unidades monetarias. Se almacenan como enteros porque el tipo counter no admite decimales.

El parámetro N se ajusta mediante la cláusula LIMIT. Al estar el monto definido como clave de clustering descendente, el motor devuelve las primeras N filas de la partición sin ordenar ni filtrar.

7. Pruebas de tolerancia a fallos

El script pruebas_fallos.py ejecuta una consulta treinta veces por cada nivel de consistencia y reporta las ejecuciones exitosas, las fallidas y las latencias. Recibe como argumento la etiqueta del escenario.

Escenario 1 — clúster completo:

```
docker exec cass1 nodetool status
python3 scripts/pruebas_fallos.py "3 nodos activos"
```

Escenario 2 — un nodo detenido:

```
docker stop cass3
sleep 40
docker exec cass1 nodetool status
python3 scripts/pruebas_fallos.py "nodo cass3 caido"
```

En este escenario uno de los nodos aparece con estado DN. Los niveles ONE y QUORUM completan las treinta ejecuciones; el nivel ALL falla en la totalidad con la excepción Unavailable, puesto que exige la confirmación de las tres réplicas.

Escenario 3 — recuperación:

```
docker start cass3
sleep 90
docker exec cass1 nodetool status
docker exec cass1 nodetool repair -pr aerolinea
python3 scripts/pruebas_fallos.py "cluster recuperado"
```

La reparación tarda alrededor de un minuto y medio y finaliza con el mensaje «Repair completed successfully». Tras ella, el nivel ALL vuelve a completar las treinta ejecuciones.

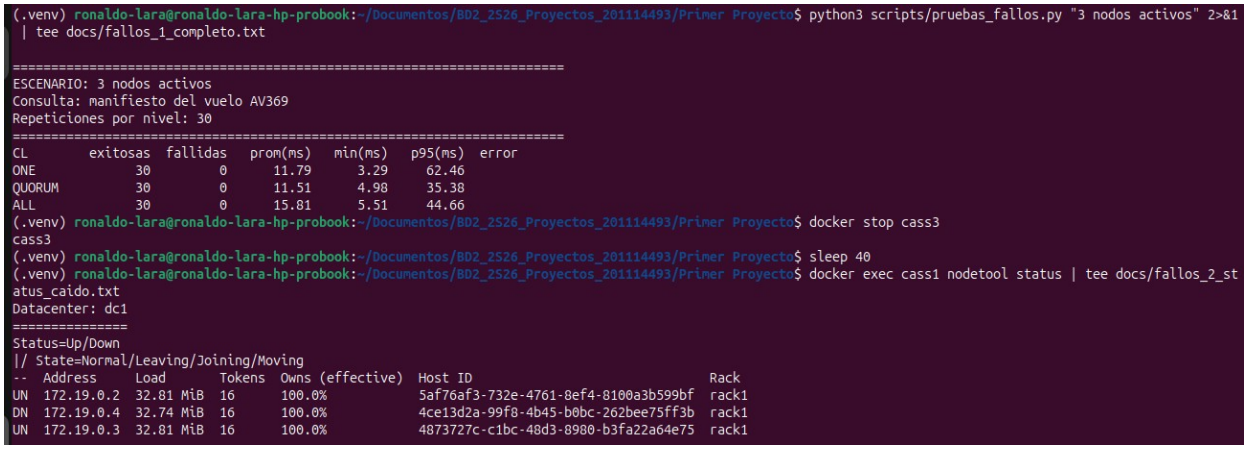


Figura 2. Escenario 1: clúster íntegro.

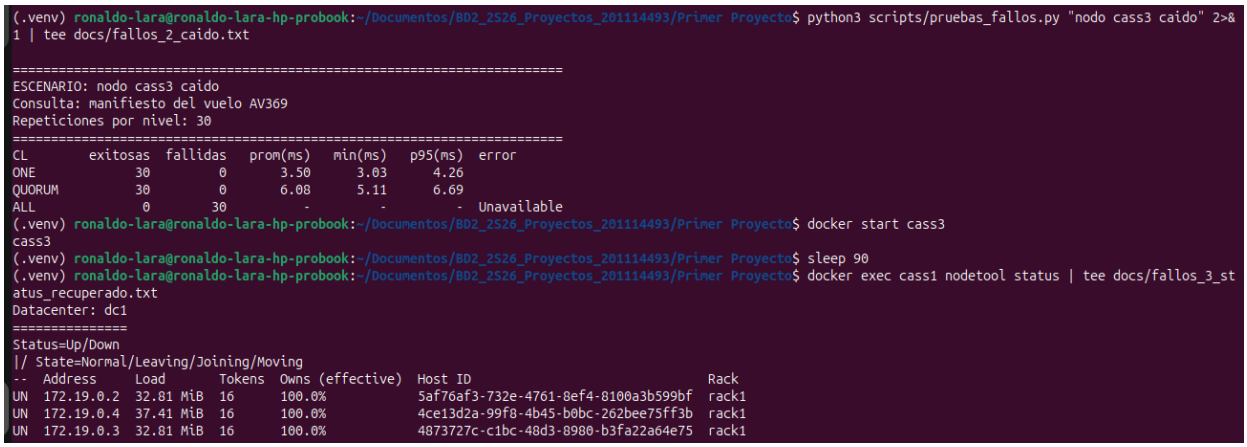


Figura 3. Escenario 2: nodo detenido; el nivel ALL falla con Unavailable.

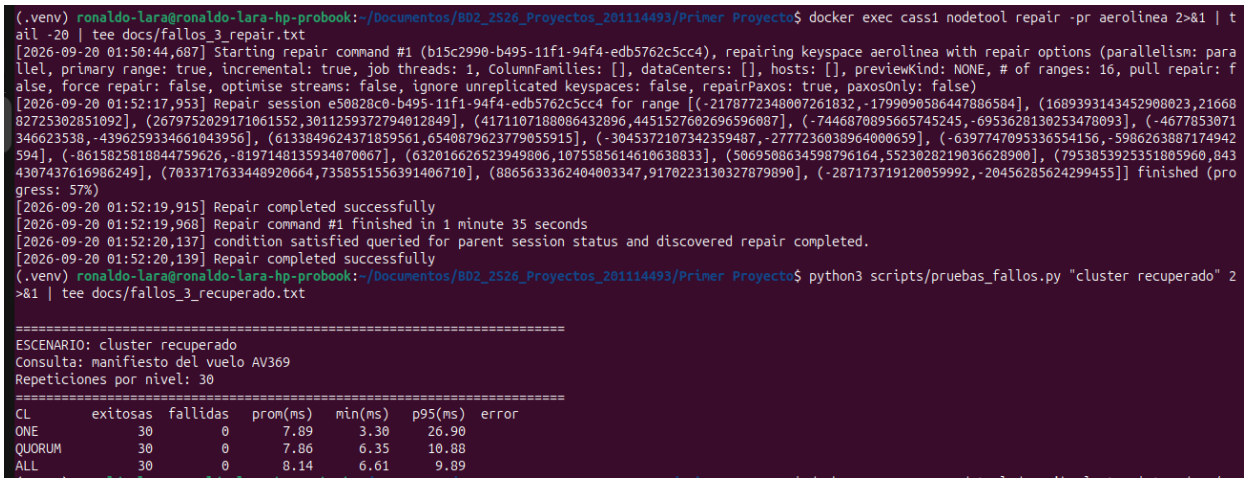


Figura 4. Escenario 3: reparación completada y clúster restablecido.

8. Operación del clúster

Acción	Comando
Detener el clúster conservando los datos	`docker compose stop`
Reanudar el clúster	`docker compose start`
Estado de los nodos	`docker exec cass1 nodetool status`
Información del clúster y esquema	`docker exec cass1 nodetool describecluster`
Consola interactiva CQL	`docker exec -it cass1 cqlsh`
Registro de un nodo	`docker logs cass2 --tail 50`
Forzar escritura a disco	`docker exec cass1 nodetool flush`
Eliminar el clúster y sus datos	`docker compose down -v`

La opción -v del último comando elimina también los volúmenes, es decir, la totalidad de los datos cargados. Para detener el clúster sin perder información debe emplearse docker compose stop.

9. Resolución de problemas comunes

9.1 Un nodo no se une al clúster

Síntoma: nodetool status muestra menos de tres nodos, o alguno permanece en estado DN.

Causa habitual: memoria insuficiente. Cada nodo reserva 512 MB de heap, de modo que el clúster completo requiere cerca de dos gigabytes además del consumo del sistema operativo.

```
free -h
docker logs cass3 --tail 50
```

Solución: cerrar aplicaciones que consuman memoria —el navegador suele ser la principal— y reiniciar el contenedor con docker restart cass3. Si el registro muestra mensajes de OutOfMemoryError, debe reducirse el valor de MAX_HEAP_SIZE en docker-compose.yml.

9.2 OperationTimedOut al crear las tablas

Síntoma: la ejecución de un archivo CQL falla con un mensaje de tiempo de espera agotado y, a veces, con la advertencia «schema version mismatch detected».

Causa: las operaciones de definición de esquema requieren que los tres nodos acuerden la misma versión, y el tiempo de espera predeterminado de cqlsh es de diez segundos.

```
docker exec -i cass1 cqlsh --request-timeout=120 -f /cql/02_tablas.cql
docker exec cass1 nodetool describecluster | grep -A3 'Schema versions'
```

Si persisten varios identificadores de esquema, debe esperarse un minuto y volver a comprobar. Si la situación no se resuelve, reiniciar los nodos secundarios con docker restart cass2 cass3 y esperar noventa segundos.

9.3 WriteTimeout durante la carga de datos

Síntoma: el script se interrumpe con el mensaje «Coordinator node timed out waiting for replica nodes' responses».

Causa: el clúster no absorbe el ritmo de escritura. Las actualizaciones de columnas counter son especialmente costosas, ya que requieren una lectura previa a la escritura.

Soluciones, en orden de efectividad:

- Liberar memoria cerrando aplicaciones; es la medida que resuelve el problema en la mayoría de los casos.
- Reducir el valor de MAX_PENDIENTES en carga_datos.py.
- Reducir el valor de TAM_BATCH.

Antes de volver a ejecutar la carga es obligatorio vaciar todas las tablas, según el procedimiento descrito en la sección 5.

9.4 Las cifras de disponibilidad o ingresos son incorrectas

Síntoma: los asientos ocupados superan la capacidad del vuelo, o los ingresos resultan desproporcionados.

Causa: el script de carga se ejecutó más de una vez sin vaciar las tablas. Las columnas de tipo counter acumulan, de modo que una segunda ejecución duplica los valores. A diferencia de las inserciones ordinarias, que sobrescriben la fila, los contadores suman.

Solución: vaciar todas las tablas y repetir la carga completa.

9.5 Permission denied al ejecutar docker

Síntoma: «permission denied while trying to connect to the Docker daemon socket».

```
sudo usermod -aG docker $USER
newgrp docker
```

La pertenencia al grupo se aplica de forma definitiva al cerrar y volver a iniciar la sesión.

9.6 SyntaxException al ejecutar cqlsh con -e

Síntoma: «line 1:0 no viable alternative at input ';'».

Causa: la cadena entre comillas comienza con un salto de línea, que el analizador de CQL no admite.

Solución: escribir la sentencia en una sola línea, o bien ejecutar cada sentencia en un comando independiente.

9.7 El puerto 9042 está ocupado

Síntoma: docker compose up falla indicando que el puerto ya está asignado.

```
sudo lsof -i :9042
```

Solución: detener el proceso que ocupa el puerto, o modificar la asignación en docker-compose.yml de 9042:9042 a 9142:9042, recordando que en ese caso los scripts de Python deben apuntar al puerto 9142.

10. Consideraciones sobre el modelo

Las siguientes particularidades del diseño conviene tenerlas presentes al operar el sistema:

- No existen operaciones JOIN. Si una consulta necesita datos de dos tablas, el modelo carece de la estructura adecuada para ese patrón de acceso.
- Los mismos datos residen en varias tablas. Una modificación debe propagarse manualmente a todas ellas; el motor no ofrece integridad referencial.
- Toda consulta debe incluir la clave de partición completa en la cláusula WHERE. En caso contrario, Cassandra exige ALLOW FILTERING, cláusula que este proyecto evita por completo.
- El ordenamiento únicamente es posible sobre claves de clustering y en el sentido declarado en la definición de la tabla.
- Las tablas con columnas de tipo counter no admiten columnas ordinarias ni TTL, razón por la cual las consultas Q1 y Q4 se resuelven con dos tablas cada una.
- Los montos monetarios de la tabla de ingresos se almacenan en centavos, dado que el tipo counter no admite valores decimales.